

go-mdtopdf-helper

Two command-line tools for keeping documentation in sync with your Markdown:

- `go-mdtopdf-helper` converts Markdown files to PDF, HTML or DOCX.
- `go-tomd` does the reverse, converting PDF, HTML and DOCX documents back to Markdown.

Perfect for maintaining PDF documentation alongside your Markdown files in Git repositories.

Features

- Markdown to PDF, HTML or DOCX, one or several formats at once
- Documents back to Markdown (`go-tomd`)
- Works out of the box: the default PDF engine, the DOCX writer and the DOCX reader are pure Go with no external dependencies
- Choice of PDF engines (pure Go, headless Chrome, or `wkhtmltopdf`)
- Convert a single file or scan a directory; choose where the output goes
- Can run as a Git pre-commit hook
- Recursive directory scanning and parallel processing
- Cross-platform support (Windows, Linux, macOS)

Installation

Download the archive for your platform from the project's GitHub releases. Each archive contains both `go-mdtopdf-helper` and `go-tomd`.

Or build from source (requires Go and, optionally, `golangci-lint` for `make lint`):

```
git clone https://github.com/Warky-Devs/go-mdtopdf-helper.git
cd go-mdtopdf-helper
make build          # writes bin/go-mdtopdf-helper and bin/go-tomd
```

go-mdtopdf-helper: Markdown to PDF, HTML and DOCX

Convert every Markdown file under the current directory to PDF:

```
go-mdtopdf-helper
```

Options

```
go-mdtopdf-helper [options]
```

Options:

```
-dir string
    Directory to scan for markdown files (default ".")
```

```
-input string
    Convert only this markdown file (instead of scanning -dir)
-output string
    With -input: output file name, extension added per format
    Without -input: directory to write results to
-recursive
    Scan directories recursively (default true)
-parallel
    Convert files in parallel (default true)
-format string
    Output formats, comma separated: pdf, html, docx (default "pdf")
-engine string
    PDF engine: gopdf, wkhtmltopdf, chromedp (default "gopdf")
-hook
    Run as git pre-commit hook
```

Output formats

Select formats with `-format`, for example `-format pdf,html,docx`. By default each file is written next to its Markdown source with the matching extension (`README.md` becomes `README.pdf`, `README.html` and `README.docx`).

- **pdf**: rendered by the selected engine (see below). A4 with 15mm margins.
- **html**: a standalone page with embedded CSS.
- **docx**: a Word document generated in pure Go (no Word or other tools needed). Supports headings, paragraphs, bold/italic/code, links, bulleted and numbered lists, code blocks, quotes, rules and tables. Images are written as `[image: alt]` text.

PDF engines

The engine only applies to PDF output. Select one with `-engine`:

Engine	Needs	Notes
gopdf (default)	nothing	Pure Go. Supports headings, paragraphs, bold/italic/code, links, lists, code blocks, quotes, rules and tables. Images are shown as <code>[image: alt]</code> .
chromedp	Chrome or Chromium	Best rendering (full HTML/CSS). Runs headless Chrome; the sandbox is disabled automatically when running as root, as in most CI containers.
wkhtmltopdf	wkhtmltopdf	Legacy engine. Looked up in the standard install locations (Windows: <code>C:\Program Files\wkhtmltopdf\bin</code> , Linux/macOS: <code>/usr/local/bin</code> , <code>/usr/bin</code> , Homebrew).

Git pre-commit hook

To run it as a Git pre-commit hook:

1. Create a file named `pre-commit` in your repository's `.git/hooks/` directory
2. Add the following content:

```
#!/bin/sh
```

```
go-mdtopdf-helper -hook
```

1. Make the hook executable:

```
chmod +x .git/hooks/pre-commit
```

When enabled as a pre-commit hook, the tool will: 1. Detect staged Markdown files 2. Ask for confirmation before conversion 3. Convert the files to the formats given by `-format` (PDF by default) 4. Automatically stage every generated file

To keep HTML or DOCX copies up to date as well, add the formats to the hook, for example `go-mdtopdf-helper -hook -format pdf,html`. The hook cannot be combined with `-input` or `-output`.

go-tomd: documents to Markdown

`go-tomd` scans a directory for `.pdf`, `.html/.htm` and `.docx` files and writes `<name>.md` next to each one.

```
go-tomd
```

Options

```
go-tomd [options]
```

Options:

- `-dir string`
Directory to scan for documents (default ".")
- `-input string`
Convert only this document (instead of scanning `-dir`); `-format` is ignored
- `-output string`
With `-input`: Markdown file name to write (.md added if missing)
Without `-input`: directory to write results to
- `-recursive`
Scan directories recursively (default true)
- `-parallel`
Convert files in parallel (default true)
- `-format string`
Input formats to convert, comma separated: docx, html, pdf (default "docx,html,pdf")
- `-force`
Overwrite Markdown files that already exist

An existing `.md` is never overwritten unless you pass `-force`, so converting `doc.html` will not clobber a `doc.md` you wrote by hand. If several documents map to the same Markdown file (for example `doc.docx` and `doc.pdf`), one is chosen in the order docx, html, pdf and a warning is printed.

How each format is read

Input	How	Fidelity
-------	-----	----------

docx	Parsed directly (no Word needed). Headings, bold/italic/code, links, nested bulleted and numbered lists, code blocks, quotes, rules and tables.	High
html	html-to-markdown , including tables and strikethrough.	High
pdf	Best effort, pure Go. PDFs have no real structure, so headings are inferred from font size, code from monospaced fonts, and lists from bullet or number prefixes. Tables and images are not recovered, and text in multi-column layouts may interleave.	Good on PDFs made by this tool, rougher on arbitrary ones

Input and output names

Both commands take `-input` and `-output`:

```
# Convert one file, choosing the output name (the extension is added per format)
go-mdtopdf-helper -input docs/guide.md -output build/manual -format pdf,html
#   -> build/manual.pdf, build/manual.html

# Scan a directory but write results elsewhere, mirroring the folder layout
go-mdtopdf-helper -dir docs -output build

# And the reverse
go-tomd -input report.docx -output notes/report      # -> notes/report.md
go-tomd -dir exports -output imported
```

- `-input` replaces the directory scan, so it cannot be combined with `-dir` or `-recursive` (or `-hook`). `go-mdtopdf-helper` expects a `.md/.markdown` file, and `go-tomd` a `.pdf`, `.html/.htm` or `.docx` file.
- With `-input`, `-output` is a file name. For `go-mdtopdf-helper` it is a base name that gets each format's extension; a name that already ends in the extension of the single requested format (`-output out.pdf -format pdf`) is used as is, and a mismatched extension is an error. For `go-tomd`, `.md` is added unless the name already ends in `.md` or `.markdown`.
- Without `-input`, `-output` is a directory, created if needed.
- Parent directories of the output are created automatically.

Project layout

<code>cmd/go-mdtopdf-helper/</code>	Markdown to PDF/HTML/DOCX CLI (flags, hook mode)
<code>cmd/go-tomd/</code>	PDF/HTML/DOCX to Markdown CLI
<code>internal/converter/</code>	File discovery, parallel conversion, git hook
<code>internal/document/</code>	HTML and DOCX writers
<code>internal/engine/</code>	Engine interface and the <code>gopdf</code> , <code>wkhtmltopdf</code> and <code>chromedp</code> backends
<code>internal/tomd/</code>	PDF, HTML and DOCX readers (to Markdown)

Contributing

Contributions are welcome! Please feel free to submit a Pull Request. Here's how you can contribute:

1. Fork the repository
2. Create your feature branch (`git checkout -b feature/AmazingFeature`)
3. Commit your changes (`git commit -m 'Add some AmazingFeature'`)
4. Push to the branch (`git push origin feature/AmazingFeature`)
5. Open a Pull Request

Development setup

1. Clone the repository
2. Install dependencies:

```
go mod download
```

3. Make your changes
4. Run the checks:

```
make all
```

Make targets: `build` (both binaries into `bin/`), `run` and `run-tomd` (pass arguments with `ARGS="..."`), `test`, `lint` (`golangci-lint`), `format` (`gofmt`), `clean`, and `all` (`format`, `lint`, `test`, `build`). Tests for the `chromedp` and `wkhtmltopdf` engines are skipped when the tool is not installed.

Dependencies

- [gopdf](#) – Pure Go PDF generation
- [chromedp](#) – Headless Chrome driver
- [go-wkhtmltopdf](#) – Go wrapper for wkhtmltopdf
- [gomarkdown](#) – Markdown parser and HTML renderer
- [pdf](#) – Pure Go PDF text extraction (`go-tomd`)
- [html-to-markdown](#) – HTML to Markdown (`go-tomd`)

License

This project is licensed under the MIT License – see the `LICENSE` file for details.

Acknowledgments

- Thanks to [Sebastiaan Klippert](#) for the `go-wkhtmltopdf` library
- Thanks to the `gomarkdown` team for their Markdown parser